

AWT(ABSTRACT WINDOW TOOLKIT)

AWT:-

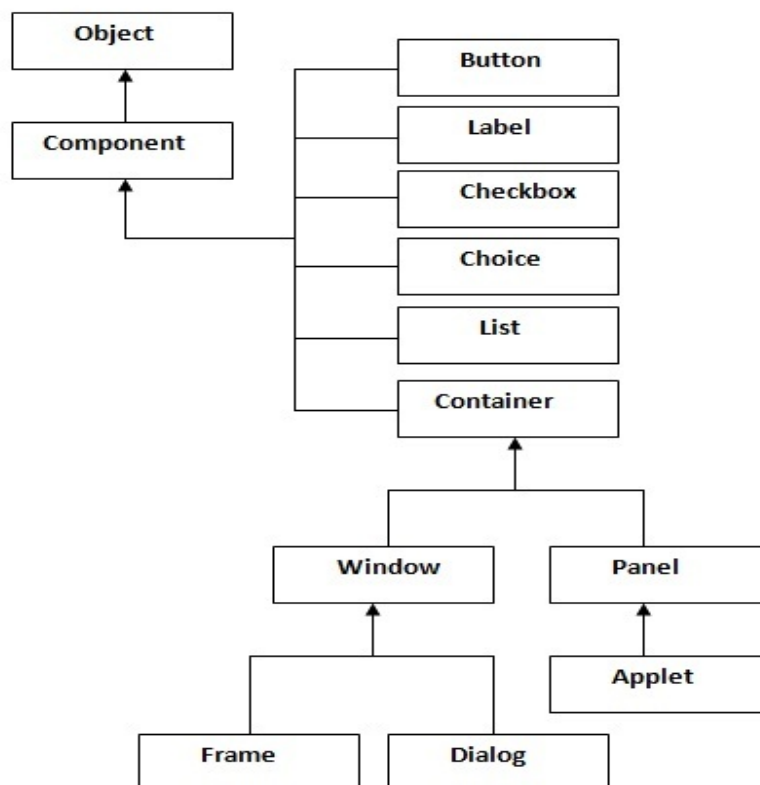
Java AWT (Abstract Windowing Toolkit) is *an API to develop GUI or window-based application in java.*

Java AWT components are platform-dependent i.e. components are displayed according to the view of operating system. AWT is heavyweight i.e. its components uses the resources of system.

The java.awt package provides classes for AWT api such as TextField, Label, TextArea, RadioButton, CheckBox, Choice, List etc.

Java AWT Hierarchy

The hierarchy of Java AWT classes are given below.



Term	Description
Component	Component is an object having a graphical representation that can be displayed on the screen and that can interact with the user. For examples buttons, checkboxes, list and scrollbars of a graphical user interface.
Container	Container object is a component that can contain other components. Components added to a container are tracked in a list. The order of the list will define the components' front-to-back stacking order within the container. If no index is specified when adding a component to a container, it will be added to the end of the list.
Panel	Panel provides space in which an application can attach any other components, including other panels.
Window	Window is a rectangular area which is displayed on the screen. In different window we can execute different program and display different data. Window provide us with multitasking environment. A window must have either a frame, dialog, or another window defined as its owner when it's constructed.
Frame	A Frame is a top-level window with a title and a border. The size of the frame includes any area designated for the border. Frame encapsulates window . It and has a title bar, menu bar, borders, and resizing corners. It can have other components like button, textfield etc.
Canvas	Canvas component represents a blank rectangular area of the screen onto which the application can draw. Application can also trap input events from the use from that

Useful Methods of Component class

Method	Description
public void add(Component c)	inserts a component on this component.
public void setSize(int width,int height)	sets the size (width and height) of the component.
public void setLayout(LayoutManager m)	defines the layout manager for the component.
public void setVisible(boolean status)	changes the visibility of the component, by default false.

AWT Controls:-

Every user interface considers the following three main aspects:

- **UI elements** : These are the core visual elements the user eventually sees and interacts with. GWT provides a huge list of widely used and common elements varying from basic to complex which we will cover in this tutorial.
- **Layouts**: They define how UI elements should be organized on the screen and provide a final look and feel to the GUI (Graphical User Interface). This part will be covered in Layout chapter.
- **Behavior**: These are events which occur when the user interacts with UI elements. This part will be covered in Event Handling chapter.

AWT UI Elements:

Following is the list of commonly used controls while designed GUI using AWT.

Sr. No.	Control & Description
1	Label A Label object is a component for placing text in a container.
2	Button This class creates a labeled button.
3	Check Box A check box is a graphical component that can be in either an on (true) or off (false) state.
4	Check Box Group The CheckboxGroup class is used to group the set of checkbox.
5	List The List component presents the user with a scrolling list of text items.
6	Text Field A TextField object is a text component that allows for the editing of a single line of text.
7	Text Area A TextArea object is a text component that allows for the editing of a multiple lines of text.
8	Choice A Choice control is used to show pop up menu of choices. Selected choice is shown on the top of the menu.
9	Canvas A Canvas control represents a rectangular area where application can draw something or can receive inputs created by user.
10	Image An Image control is superclass for all image classes representing graphical images.
11	Scroll Bar A Scrollbar control represents a scroll bar component in order to enable user to select from range of values.
12	Dialog A Dialog control represents a top-level window with a title and a border used to take some form of input from the user.
13	File Dialog A FileDialog control represents a dialog window from which the user can select a file.

AWT Layouts:-

Layout means the arrangement of components within the container. In other way we can say that placing the components at a particular position within the container. The task of laying out the controls is done automatically by the Layout Manager.

Layout Manager

The layout manager automatically positions all the components within the container. If we do not use layout manager then also the components are positioned by the default layout manager. It is possible to layout the controls by hand but it becomes very difficult because of the following two reasons.

- It is very tedious to handle a large number of controls within the container.

- Oftenly the width and height information of a component is not given when we need to arrange them.

Java provide us with various layout manager to position the controls. The properties like size, shape and arrangement varies from one layout manager to other layout manager. When the size of the applet or the application window changes the size, shape and arrangement of the components also changes in response i.e. the layout managers adapt to the dimensions of appletviewer or the application window.

The layout manager is associated with every Container object. Each layout manager is an object of the class that implements the LayoutManager interface.

Following are the interfaces defining functionalities of Layout Managers.

Sr. No.	Interface & Description
1	LayoutManager The LayoutManager interface declares those methods which need to be implemented by the class whose object will act as a layout manager.
2	LayoutManager2 The LayoutManager2 is the sub-interface of the LayoutManager. This interface is for those classes that know how to layout containers based on layout constraint object.

AWT Layout Manager Classes:

Following is the list of commonly used controls while designed GUI using AWT.

Sr. No.	LayoutManager & Description
1	BorderLayout The BorderLayout arranges the components to fit in the five regions: east, west, north, south and center.
2	CardLayout The CardLayout object treats each component in the container as a card. Only one card is visible at a time.
3	FlowLayout The FlowLayout is the default layout. It layouts the components in a directional flow.
4	GridLayout The GridLayout manages the components in form of a rectangular grid.
5	GridBagLayout This is the most flexible layout manager class. The object of GridBagLayout aligns the component vertically, horizontally or along their baseline without requiring the components of same size.

AWT Graphics Class:-

The Graphics class is the abstract super class for all graphics contexts which allow an application to draw onto components that can be realized on various devices, or onto off-screen images as well.

A Graphics object encapsulates all state information required for the basic rendering operations that Java supports. State information includes the following properties.

- The Component object on which to draw.
- A translation origin for rendering and clipping coordinates.
- The current clip.
- The current color.
- The current font.
- The current logical pixel operation function.
- The current XOR alternation color

Class declaration

Following is the declaration for **java.awt.Graphics** class:

```
public abstract class Graphics
    extends Object
```

Class constructors

S.N.	Constructor & Description
1	Graphics() () Constructs a new Graphics object.

Class methods

S.N.	Method & Description
1	abstract void clearRect(int x, int y, int width, int height) Clears the specified rectangle by filling it with the background color of the current drawing surface.
2	abstract void clipRect(int x, int y, int width, int height) Intersects the current clip with the specified rectangle.
3	abstract void copyArea(int x, int y, int width, int height, int dx, int dy) Copies an area of the component by a distance specified by dx and dy.
4	abstract Graphics create() Creates a new Graphics object that is a copy of this Graphics object.
5	Graphics create(int x, int y, int width, int height) Creates a new Graphics object based on this Graphics object, but with a new translation and clip area.
6	abstract void dispose() Disposes of this graphics context and releases any system resources that it is using.
7	void draw3DRect(int x, int y, int width, int height, boolean raised) Draws a 3-D highlighted outline of the specified rectangle.
8	abstract void drawArc(int x, int y, int width, int height, int startAngle, int arcAngle) Draws the outline of a circular or elliptical arc covering the specified rectangle.
9	void drawBytes(byte[] data, int offset, int length, int x, int y) Draws the text given by the specified byte array, using this graphics context's current font and color.
10	void drawChars(char[] data, int offset, int length, int x, int y) Draws the text given by the specified character array, using this graphics context's current font and color.
11	abstract boolean drawImage(Image img, int x, int y, Color bgcolor, ImageObserver observer) Draws as much of the specified image as is currently available.

12	abstract boolean drawImage(Image img, int x, int y, ImageObserver observer) Draws as much of the specified image as is currently available.
13	abstract boolean drawImage(Image img, int x, int y, int width, int height, Color bgcolor, ImageObserver observer) Draws as much of the specified image as has already been scaled to fit inside the specified rectangle.
14	abstract boolean drawImage(Image img, int x, int y, int width, int height, ImageObserver observer) Draws as much of the specified image as has already been scaled to fit inside the specified rectangle.
15	abstract boolean drawImage(Image img, int dx1, int dy1, int dx2, int dy2, int sx1, int sy1, int sx2, int sy2, Color bgcolor, ImageObserver observer) Draws as much of the specified area of the specified image as is currently available, scaling it on the fly to fit inside the specified area of the destination drawable surface.
16	abstract boolean drawImage(Image img, int dx1, int dy1, int dx2, int dy2, int sx1, int sy1, int sx2, int sy2, ImageObserver observer) Draws as much of the specified area of the specified image as is currently available, scaling it on the fly to fit inside the specified area of the destination drawable surface.
17	abstract void drawLine(int x1, int y1, int x2, int y2) Draws a line, using the current color, between the points (x1, y1) and (x2, y2) in this graphics context's coordinate system.
18	abstract void drawOval(int x, int y, int width, int height) Draws the outline of an oval.
19	abstract void drawPolygon(int[] xPoints, int[] yPoints, int nPoints) Draws a closed polygon defined by arrays of x and y coordinates.
20	void drawPolygon(Polygon p) Draws the outline of a polygon defined by the specified Polygon object.
21	abstract void drawPolyline(int[] xPoints, int[] yPoints, int nPoints) Draws a sequence of connected lines defined by arrays of x and y coordinates.
22	void drawRect(int x, int y, int width, int height) Draws the outline of the specified rectangle.
23	abstract void drawRoundRect(int x, int y, int width, int height, int arcWidth, int arcHeight) Draws an outlined round-cornered rectangle using this graphics context's current color.
24	abstract void drawString(AttributedCharacterIterator iterator, int x, int y) Renders the text of the specified iterator applying its attributes in accordance with the specification of the TextAttribute class.
25	abstract void drawString(String str, int x, int y) Draws the text given by the specified string, using this graphics context's current font and color.
26	void fill3DRect(int x, int y, int width, int height, boolean raised) Paints a 3-D highlighted rectangle filled with the current color.
27	abstract void fillArc(int x, int y, int width, int height, int startAngle, int arcAngle) Fills a circular or elliptical arc covering the specified rectangle.
28	abstract void fillOval(int x, int y, int width, int height) Fills an oval bounded by the specified rectangle with the current color.
29	abstract void fillPolygon(int[] xPoints, int[] yPoints, int nPoints) Fills a closed polygon defined by arrays of x and y coordinates.
30	void fillPolygon(Polygon p) Fills the polygon defined by the specified Polygon object with the graphics context's current color.
31	abstract void fillRect(int x, int y, int width, int height) Fills the specified rectangle.
32	abstract void fillRoundRect(int x, int y, int width, int height, int arcWidth, int arcHeight) Fills the specified rounded corner rectangle with the current color.

33	void finalize() Disposes of this graphics context once it is no longer referenced.
34	abstract Shape getClip() Gets the current clipping area.
35	abstract Rectangle getClipBounds() Returns the bounding rectangle of the current clipping area.
36	Rectangle getClipBounds(Rectangle r) Returns the bounding rectangle of the current clipping area.
37	Rectangle getClipRect() Deprecated. As of JDK version 1.1, replaced by getClipBounds().
38	abstract Color getColor() Gets this graphics context's current color.
39	abstract Font getFont() Gets the current font.
40	FontMetrics getFontMetrics() Gets the font metrics of the current font.
41	abstract FontMetrics getFontMetrics(Font f) Gets the font metrics for the specified font.
42	boolean hitClip(int x, int y, int width, int height) Returns true if the specified rectangular area might intersect the current clipping area.
43	abstract void setClip(int x, int y, int width, int height) Sets the current clip to the rectangle specified by the given coordinates.
44	abstract void setClip(Shape clip) Sets the current clipping area to an arbitrary clip shape.
45	abstract void setColor(Color c) Sets this graphics context's current color to the specified color.
46	abstract void setFont(Font font) Sets this graphics context's font to the specified font.
47	abstract void setPaintMode() Sets the paint mode of this graphics context to overwrite the destination with this graphics context's current color.
48	abstract void setXORMode(Color c1) Sets the paint mode of this graphics context to alternate between this graphics context's current color and the new specified color.
49	String toString() Returns a String object representing this Graphics object's value.
50	abstract void translate(int x, int y) Translates the origin of the graphics context to the point (x, y) in the current coordinate system.

AWT Example:-

To create simple awt example, you need a frame. There are two ways to create a frame in AWT.

- By extending Frame class (inheritance)
- By creating the object of Frame class (association)

Simple example of AWT by inheritance

```
import java.awt.*;

class First extends Frame

{

    First()

    {

        Button b=new Button("click me");

        b.setBounds(30,100,80,30);// setting button position


        add(b);//adding button into frame

        setSize(300,300);//frame size 300 width and 300 height

        setLayout(null);//no layout manager

        setVisible(true);//now frame will be visible, by default not visible

    }

    public static void main(String args[])

    {

        First f=new First();

    }

}
```

Simple example of AWT by association

```
import java.awt.*;

class First2 {

    First2(){

        Frame f=new Frame();
```



```
Button b=new Button("click me");
```

```
b.setBounds(30,50,80,30);
```

```
f.add(b);
```

```
f.setSize(300,300);
```

```
f.setLayout(null);
```

```
f.setVisible(true);
```

```
}
```

```
public static void main(String args[]){
```

```
First2 f=new First2();
```

```
}}
```

Output:-



Swing

Java Swing:-

Java Swing tutorial is a part of Java Foundation Classes (JFC) that is *used to create window-based applications*. It is built on the top of AWT (Abstract Windowing Toolkit) API and entirely written in java.

Unlike AWT, Java Swing provides platform-independent and lightweight components.

The javax.swing package provides classes for java swing API such as JButton, JTextField, JTextArea, JRadioButton, JCheckbox, JMenu, JColorChooser etc.

Difference between AWT and Swing

There are many differences between java awt and swing that are given below.

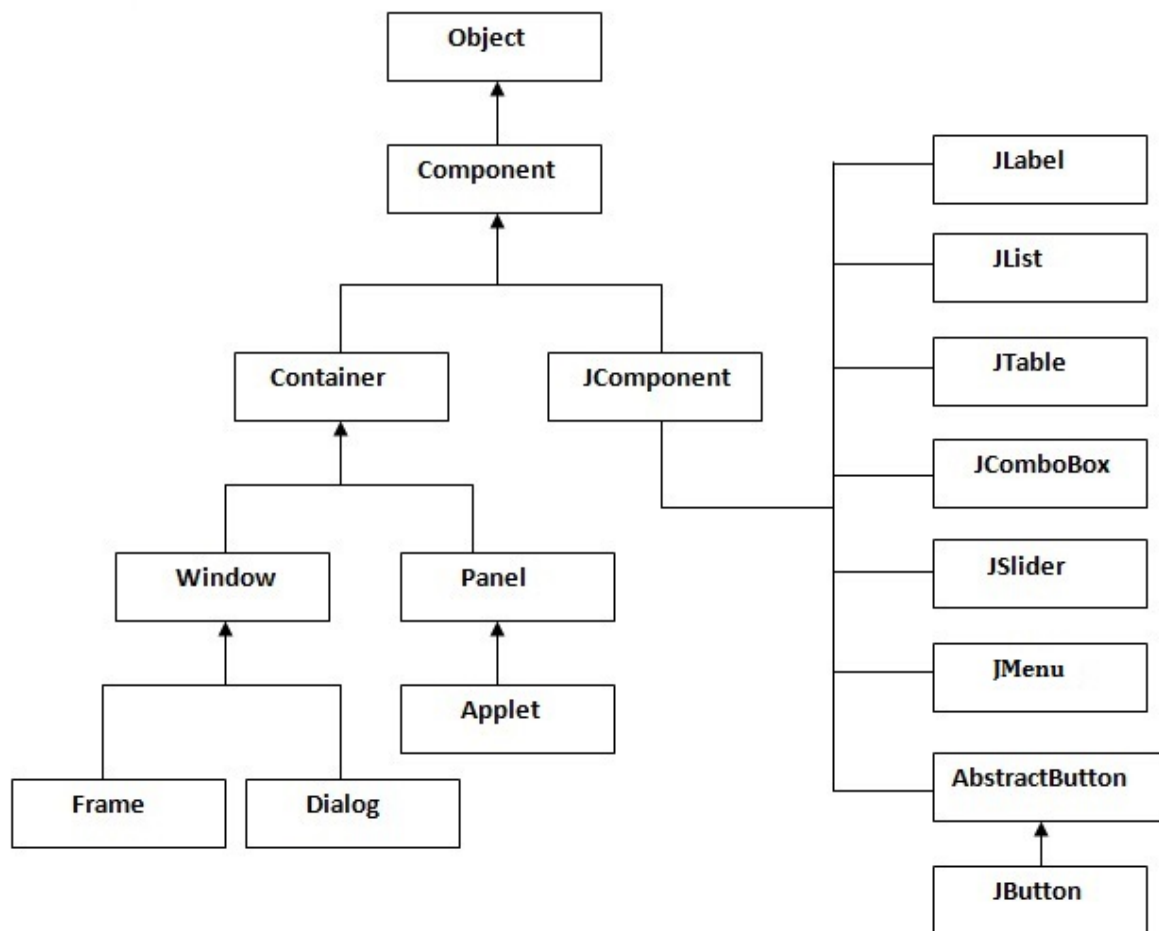
No.	Java AWT	Java Swing
1)	AWT components are platform-dependent .	Java swing components are platform-independent .
2)	AWT components are heavyweight .	Swing components are lightweight .
3)	AWT doesn't support pluggable look and feel .	Swing supports pluggable look and feel .
4)	AWT provides less components than Swing.	Swing provides more powerful components such as tables, lists, scrollpanes, colorchooser, tabbedpane etc.
5)	AWT doesn't follows MVC (Model View Controller) where model represents data, view represents presentation and controller acts as an interface between model and view.	Swing follows MVC .

JFC

The Java Foundation Classes (JFC) are a set of GUI components which simplify the development of desktop applications.

Hierarchy of Java Swing classes

The hierarchy of java swing API is given below.



Commonly used Methods of Component class

The methods of Component class are widely used in java swing that are given below.

Method	Description
public void add(Component c)	add a component on another component.
public void setSize(int width,int height)	sets size of the component.
public void setLayout(LayoutManager m)	sets the layout manager for the component.
public void setVisible(boolean b)	sets the visibility of the component. It is by default false.

Java Swing Examples

There are two ways to create a frame:

- By creating the object of Frame class (association)
- By extending Frame class (inheritance)

We can write the code of swing inside the main(), constructor or any other method.

Simple Java Swing Example

Let's see a simple swing example where we are creating one button and adding it on the JFrame object inside the main() method.

File: FirstSwingExample.java

```
import javax.swing.*.*;

public class FirstSwingExample {

    public static void main(String[] args) {

        JFrame f=new JFrame();//creating instance of JFrame

        JButton b=new JButton("click");//creating instance of JButton

        b.setBounds(130,100,100, 40);//x axis, y axis, width, height

        f.add(b);//adding button in JFrame

        f.setSize(400,500);//400 width and 500 height

        f.setLayout(null);//using no layout managers

        f.setVisible(true);//making the frame visible

    }

}
```



JButton class:

The JButton class is used to create a button that have platform-independent implementation.

Commonly used Constructors:

- **JButton():** creates a button with no text and icon.
- **JButton(String s):** creates a button with the specified text.
- **JButton(Icon i):** creates a button with the specified icon object.

Commonly used Methods of AbstractButton class:

- 1) **public void setText(String s):** is used to set specified text on button.
- 2) **public String getText():** is used to return the text of the button.
- 3) **public void setEnabled(boolean b):** is used to enable or disable the button.
- 4) **public void setIcon(Icon b):** is used to set the specified Icon on the button.
- 5) **public Icon getIcon():** is used to get the Icon of the button.
- 6) **public void setMnemonic(int a):** is used to set the mnemonic on the button.
- 7) **public void addActionListener(ActionListener a):** is used to add the action listener to this object.

JRadioButton class

The JRadioButton class is used to create a radio button. It is used to choose one option from multiple options. It is widely used in exam systems or quiz.

It should be added in ButtonGroup to select one radio button only.

Commonly used Constructors of JRadioButton class:

- **JRadioButton():** creates an unselected radio button with no text.
- **JRadioButton(String s):** creates an unselected radio button with specified text.
- **JRadioButton(String s, boolean selected):** creates a radio button with the specified text and selected status.

Commonly used Methods of AbstractButton class:

- 1) **public void setText(String s):** is used to set specified text on button.
- 2) **public String getText():** is used to return the text of the button.
- 3) **public void setEnabled(boolean b):** is used to enable or disable the button.
- 4) **public void setIcon(Icon b):** is used to set the specified Icon on the button.
- 5) **public Icon getIcon():** is used to get the Icon of the button.
- 6) **public void setMnemonic(int a):** is used to set the mnemonic on the button.
- 7) **public void addActionListener(ActionListener a):** is used to add the action listener to this object.

JComboBox class:

The JComboBox class is used to create the combobox (drop-down list). At a time only one item can be selected from the item list.

Commonly used Constructors of JComboBox class:

JComboBox()

JComboBox(Object[] items)

JComboBox(Vector<?> items)

Commonly used methods of JComboBox class:

- 1) **public void addItem(Object anObject):** is used to add an item to the item list.
- 2) **public void removeItem(Object anObject):** is used to delete an item to the item list.

3) **public void removeAllItems():** is used to remove all the items from the list.

4) **public void setEditable(boolean b):** is used to determine whether the JComboBox is editable.

5) **public void addActionListener(ActionListener a):** is used to add the ActionListener.

6) **public void addItemListener(ItemListener i):** is used to add the ItemListener.

JLabel:-

The class **JLabel** can display either text, an image, or both. Label's contents are aligned by setting the vertical and horizontal alignment in its display area. By default, labels are vertically centered in their display area. Text-only labels are leading edge aligned, by default; image-only labels are horizontally centered, by default.

Class constructors

S.N.	Constructor & Description
1	JLabel() Creates a JLabel instance with no image and with an empty string for the title.
2	JLabel(Icon image) Creates a JLabel instance with the specified image.
3	JLabel(Icon image, int horizontalAlignment) Creates a JLabel instance with the specified image and horizontal alignment.
4	JLabel(String text) Creates a JLabel instance with the specified text.
5	JLabel(String text, Icon icon, int horizontalAlignment) Creates a JLabel instance with the specified text, image, and horizontal alignment.
6	JLabel(String text, int horizontalAlignment) Creates a JLabel instance with the specified text and horizontal alignment.

ImageIcon:-

The class **ImageIcon** is an implementation of the Icon interface that paints Icons from Images.

Class constructors

S.N.	Constructor & Description
1	ImageIcon() Creates an uninitialized image icon.
2	ImageIcon(byte[] imageData) Creates an ImageIcon from an array of bytes which were read from an image file containing a supported image format, such as GIF, JPEG, or (as of 1.3) PNG.
3	ImageIcon(byte[] imageData, String description) Creates an ImageIcon from an array of bytes which were read from an image file containing a supported image format, such as GIF, JPEG, or (as of 1.3) PNG.
4	ImageIcon(Image image) Creates an ImageIcon from an image object.
	ImageIcon(Image image, String description)

- 5 Creates an ImageIcon from the image.
- 6 **ImageIcon(String filename)**
Creates an ImageIcon from the specified file.
- 7 **ImageIcon(String filename, String description)**
Creates an ImageIcon from the specified file.
- 8 **ImageIcon(URL location)**
Creates an ImageIcon from the specified URL.
- 9 **ImageIcon(URL location, String description)**
Creates an ImageIcon from the specified URL.

JTextField:-

The class **JTextField** is a component which allows the editing of a single line of text.

Class constructors

S.N.	Constructor & Description
1	JTextField() Constructs a new TextField.
2	JTextField(Document doc, String text, int columns) Constructs a new JTextField that uses the given text storage model and the given number of columns.
3	JTextField(int columns) Constructs a new empty TextField with the specified number of columns.
4	JTextField(String text) Constructs a new TextField initialized with the specified text.
5	JTextField(String text, int columns) Constructs a new TextField initialized with the specified text and columns.

JScrollBar:-

The class **JScrollBar** is an implementation of scrollbar.

Class constructors

S.N.	Constructor & Description
1	JScrollBar() Creates a vertical scrollbar with the initial values.
2	JScrollBar(int orientation) Creates a scrollbar with the specified orientation and the initial values.
3	JScrollBar(int orientation, int value, int extent, int min, int max) Creates a scrollbar with the specified orientation, value, extent, minimum, and maximum.

JTable:

The JTable class is used to display the data on two dimensional tables of cells.

Commonly used Constructors of JTable class:

- **JTable():** creates a table with empty cells.
- **JTable(Object[][] rows, Object[] columns):** creates a table with the specified data.

Example of JTable class:

```
1. import javax.swing.*;
2. public class MyTable {
3.     JFrame f;
4.     MyTable(){
5.         f=new JFrame();
6.         String data[][]={ {"101","Amit","670000"},
7.             {"102","Jai","780000"},
8.                 i. {"101","Sachin","700000"}};
9.         String column[]={"ID","NAME","SALARY"};
10.        JTable jt=new JTable(data,column);
11.        JScrollPane sp=new JScrollPane(jt);
12.        f.add(sp);
13.        f.setSize(300,400);
14.        // f.setLayout(null);
15.        f.setVisible(true);
16.    }
17.    public static void main(String[] args) {
18.        new MyTable();
19.    }
20. }
```

Example Of Swing:-

```
1. import java.awt.*;
2. import java.awt.event.*;
3. import javax.swing.*;
4. import java.sql.*;
5. import java.io.*;
6. class regn extends JFrame implements ActionListener,ItemListener
7. {
8.     JLabel nl,b,c,d,k,e,brn,hob,log;
9.     ImageIcon m;
10.    JButton i,j;
11.    Container cn;
12.    Font p,o;
13.    JTextField nt,r,s,t,u,v,li;
14.    JPasswordField pr;
15.    Connection con;
16.    Statement st;
17.    ResultSet rs;
18.    JRadioButton ml,fl;
19.    JCheckBox pl,re,ch;
```

```

20. JComboBox jcb;
21. ButtonGroup bg;
22. String nm,l,pw,phn,ge,br,hb="";
23. int rol;
24. public regn()
25. {
26. try
27. {
28. Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");
29. }
30. catch(ClassNotFoundException aa)
31. {
32. System.out.println(aa);
33. }
34. m=new ImageIcon("book.jpg");
35. k=new JLabel(m);
36. p=new Font("Forte",Font.ITALIC,25);
37. o=new Font("Forte",Font.ITALIC,30);
38. setTitle("New Member");
39. cn=getContentPane();
40. cn.setLayout(null);
41. nl=new JLabel("Name"); nl.setFont(o); nl.setBounds(100,100,100,30);
42. cn.add(nl);
43. nt=new JTextField(30); nt.setBounds(400,100,150,30);
44. cn.add(nt);
    a. log=new JLabel("Login Id"); log.setFont(o); log.setBounds(100,150,150,30);
45. cn.add(log);
46. li=new JTextField(30); li.setBounds(400,150,150,30);
47. cn.add(li);
48. b=new JLabel("Password"); b.setFont(o); b.setBounds(100,200,150,30);
49. cn.add(b);
50. pr=new JPasswordField(30); pr.setBounds(400,200,150,30);
51. cn.add(pr);
52. e=new JLabel("Roll No"); e.setFont(o); e.setBounds(100,300,150,30);
53. cn.add(e);
54. v=new JTextField(30); v.setBounds(400,300,150,30);
55. cn.add(v);
56. c=new JLabel("Phn. No."); c.setFont(o); c.setBounds(100,400,150,30);
57. cn.add(c);
58. r=new JTextField(30); r.setBounds(400,400,150,30);
59. cn.add(r);
60. d=new JLabel("Gender"); d.setFont(o); d.setBounds(100,500,150,30);
61. cn.add(d);
62. ml=new JRadioButton("Male");
63. ml.setBounds(300,500,200,30);
64. ml.setFont(o);
65. ml.addActionListener(this);
66. cn.add(ml);
67. fl=new JRadioButton("Female");
68. fl.setBounds(550,500,300,30);
69. fl.setFont(o);
70. fl.addActionListener(this);
71. cn.add(fl);
72. ButtonGroup bg=new ButtonGroup();
73. bg.add(fl);
74. bg.add(ml);

```

```

75. brn=new JLabel("Branch"); brn.setFont(o); brn.setBounds(100,550,150,30);
76. cn.add(brn);
77. jcb=new JComboBox();
78. jcb.addItem("CSE");
79. jcb.addItem("MCA");
80. jcb.addItem("CIVIL");
81. jcb.addItem("ME");
82. jcb.addItem("CE");
83. jcb.setBounds(300,550,150,30);
84. jcb.addItemListener(this);
85. cn.add(jcb);
86. hob=new JLabel("Hobbies"); hob.setFont(o); hob.setBounds(100,600,150,30);
87. cn.add(hob);
88. pl=new JCheckBox("Play");
89. pl.setBounds(300,600,100,30);
90. pl.addItemListener(this);
91. cn.add(pl);
92. ch=new JCheckBox("Chat");
93. ch.setBounds(450,600,100,30);
94. ch.addItemListener(this);
95. cn.add(ch);
96. re=new JCheckBox("Read");
97. re.setBounds(600,600,100,30);
98. re.addItemListener(this);
99. cn.add(re);
100.j=new JButton("Submit"); j.setFont(p); j.setBounds(100,700,150,30);
    j.addActionListener(this);
101.cn.add(j);
102.i=new JButton("Reset"); i.setFont(p); i.setBounds(300,700,150,30);
    i.addActionListener(this);
103.cn.add(i);
104.k.setBounds(0,0,1200,1000);
105.cn.add(k);
106.setSize(1200,1000);
107.setVisible(true);
108.}
109.public void itemStateChanged(ItemEvent ie)
110.{
111.br=(String)jcb.getSelectedItem();
112.if(ie.getSource()==pl)
113.{
114.hb+=pl.getText() + "/";
115.}
116.if(ie.getSource()==ch)
117.{
118.hb+=ch.getText() + "/";
119.}
120.if(ie.getSource()==re)
121.{
122.hb+=re.getText();
123.}
124.}
125.public void actionPerformed(ActionEvent ae)
126.{
127.if(ae.getSource()==ml)
128.{

```

```

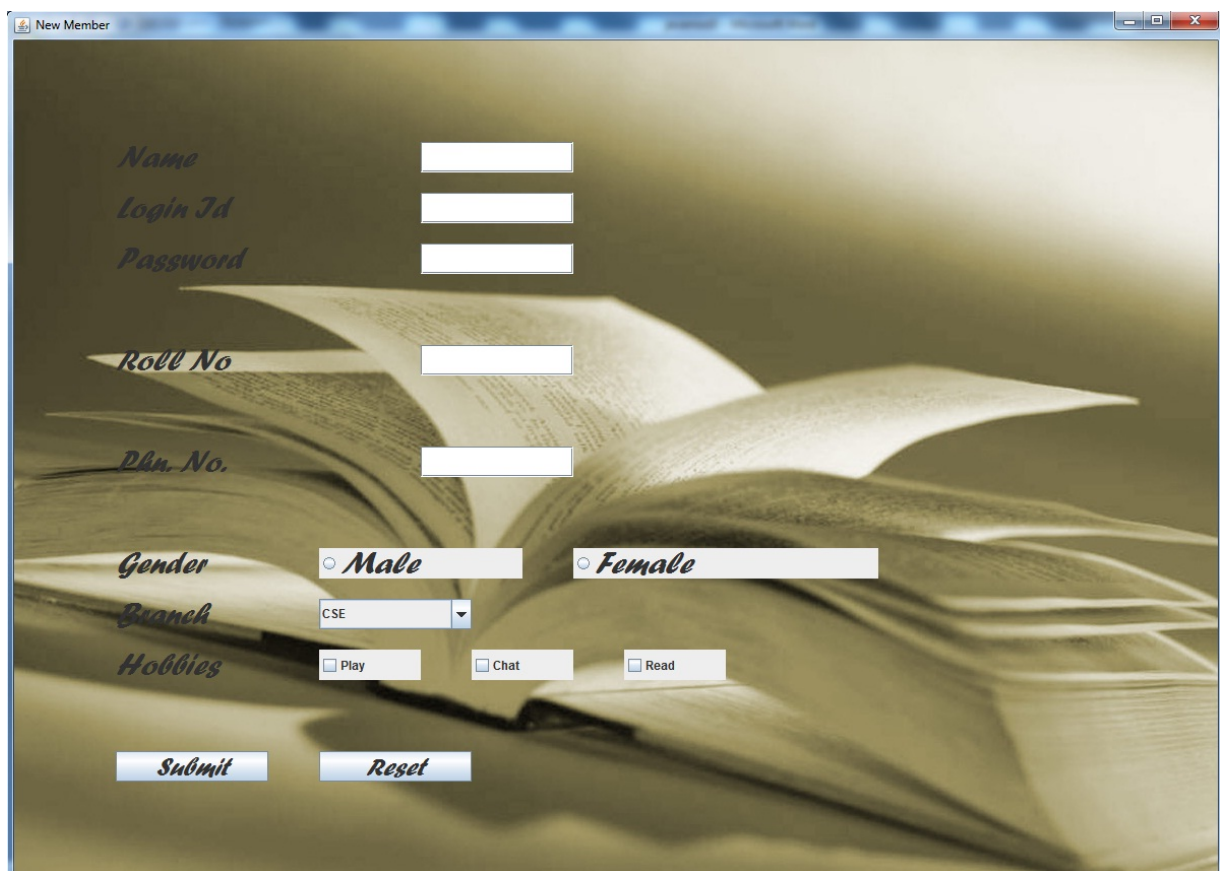
129.ge=ml.getText();
130.}
131.if(ae.getSource()==fl)
132.{
133.ge=fl.getText();
134.}
135.if(ae.getSource()==i)
136.{
137.nt.setText("");
138.li.setText("");
139.pr.setText("");
140.v.setText("");
141.r.setText("");
142.}
143.if(ae.getSource()==j)
144.{
145.nm=nt.getText();
146.l=li.getText();
147.pw=pr.getText();
148.rol=Integer.parseInt(v.getText());
149.phn=r.getText();
150.//System.out.println(nm + ":" + l + ":" + pw + ":" + rol + ":" + phn);
151.//System.out.println( br);
152.//System.out.println( ge);
153.//System.out.println( hb);
154.try
155.{
156.con=DriverManager.getConnection("jdbc:odbc:simple");
157.st=con.createStatement();
158.rs=st.executeQuery("select * from users where (Loginid='"+l+"'");
159.if(rs.next())
160.{
161.//System.out.println( "data insert failed");
162.JOptionPane.showMessageDialog(null,"Loginid already  already
    exist","ERROR",JOptionPane.ERROR_MESSAGE);
163.}
164.else
165.{
166.st.executeUpdate("insert into users (Name,Loginid>Password,Rollno,Phoneno,Gender,Branch,Hobbies)
    values('"+nm+"','"+l+"','"+pw+"','"+rol+"','"+phn+"','"+ge+"','"+br+"','"+hb+"')");
167.//System.out.println( "data inserted successfully");
168.JOptionPane.showMessageDialog(null,"registered
    successfully","Message",JOptionPane.PLAIN_MESSAGE);
169.setVisible(false);
170.new home();
171.}
172.}catch(SQLException sae)
173.{
174.System.out.println(sae);
175.}
176.}

177.}
178.}
179./*class regd
180.{

```

```
181. public static void main(String args[])
182. {
183. new regn();
184. }
185. }*/
```

Output:-



New Member

Name

Login Id

Password

Roll No

Phn. No.

Gender ☐ *Male* ☐ *Female*

Branch

Hobbies ☐ Play ☐ Chat ☐ Read